United States Patent	12400019
Kind Code	B1
Date of Patent	August 26, 2025
Inventor(s)	Papamanthou; Charalampos et al.

Systems and methods for cryptographically verifying queries

Abstract

Methods and systems are provided for cryptographically verifying SQL queries. The method comprises: computing a succinct digest for a table stored in a database; receiving a query in structured query language (SQL) for querying data in the table; decomposing the query into one or more basic queries; computing one proof per basic query in parallel and in a distributed fashion; and verifying the query by verifying the previously computed proofs in parallel, and checking for consistency of their public statements, according to the previously executed query decomposition.

Inventors: **Papamanthou; Charalampos (Fairfield, CT), Srinivasan; Shravan (Gaithersburg, MD), Fitzgerald; Joshua (Fort Wayne, IN), Bunner; Nathaniel (Carrboro, NC), Salumets; Andrus (Amsterdam, NL), Gailly; Nicolas (Paris, FR), Hishon-Rezaizadeh; Ismael (New York, NY)**

Applicant: **Lagrange Labs Inc.** (New York, NY)

Family ID: **1000008243424**

Assignee: **LAGRANGE LABS INC. (New York, NY)**

Appl. No.: **18/927514**

Filed: **October 25, 2024**

Foreign Application Priority Data

GR	20240100620	Sep. 09, 2024

Related U.S. Application Data

continuation parent-doc WO PCT/US2024/050368 20241008 PENDING child-doc US 18927514

Publication Classification

Int. Cl.: **G06F21/62** (20130101); **G06F16/28** (20190101); **G06F21/60** (20130101)

U.S. Cl.:

CPC **G06F21/6227** (20130101); **G06F16/284** (20190101); **G06F21/602** (20130101);

Field of Classification Search

CPC: G06F (21/6227); G06F (16/284); G06F (21/602)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
9049185	12/2014	Papadopoulos	N/A	H04L 63/123
11829486	12/2022	Lambotte	N/A	N/A
2011/0225429	12/2010	Papamanthou	713/189	G06F 16/2246
2024/0154812	12/2023	Sun et al.	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
WO-2023072955	12/2022	WO	N/A

OTHER PUBLICATIONS

Xiling Li et al., “ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs”, PVLDB, 16(8): 1804-1816, Apr. 1, 2023 (Year: 2023). cited by examiner

Howard Wu et al., “DIZK: A Distributed Zero Knowledge Proof System”, 27th USENIX Security Symposium: 675-692, Aug. 2018 (Year: 2018). cited by examiner

Zerui Cheng et al., “zkBridge: Trustless Cross-chain Bridges Made Practical”, Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security: 3003-3017, Nov. 2022 (Year: 2022). cited by examiner

Sanjam Garg et al., “zkSaaS: Zero-Knowledge SNARKs as a Service”, 32nd USENIX Security Symposium: 4427-4444, Aug. 2023 (Year: 2023). cited by examiner

Howard Wu et al., “DIZK: A Distributed Zero Knowlegde Proof System”, 27th USENIX Security Symposium: 675-692, Aug. 2018 (Year: 2018). cited by examiner

Xiling Li et al., “ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowlegde Proofs”, PVLDB, 16(8): 1804-1816, Apr. 1, 2023 (Year: 2023). cited by examiner

Yupeng Zhang et al., “vSQL: Verifying Arbitrary SQL Queries over Dynamic Outsourced Databases”, 2017 IEEE Symposium on Security and Privacy (SP) (Year: 2017). cited by examiner

Clarke, Dwaine et al. Incremental multiset hash functions and their application to memory integrity checking. In: Advances in Cryptology—ASIACRYPT. Springer 2894:188-207 (2003). cited by applicant

Co-pending U.S. Appl. No. 18/661,204, inventors Papamanthou; Charalampos et al., filed May 10, 2024. cited by applicant

Groth, Jens. On the size of pairing-based non-interactive arguments. In: Advances in Cryptology—EUROCRYPT. Springer 9666:305-326 (2016). cited by applicant

Li, Xiling et al. Zksql: Verifiable and efficient query evaluation with zero-knowledge proofs. Proceedings of the VLDB Endowment 16(8):1804-1816 (2023). cited by applicant

Merkle, Ralph C. et al. Certified Digital Signature. In Advances in Cryptology—CRYPTO '89. Springer 435:218-238 (1989). cited by applicant

Papamanthou, Charalampos et al. Reckle Trees: Updatable Merkle Batch Proofs with Applications. Cryptology ePrint Archive:1-14 (2024). cited by applicant
Plonky2: Fast Recursive Arguments with PLONK and FRI. GitHub, Sep. 7, 2022. Available at URL:https://github.com/0xPolygonZero/plonky2/blob/main/plonky2/plonky2.pdf pp. 1-16. cited by applicant
Tamassia, Roberto. Authenticated data structures. Algorithms—ESA 2003: 11th Annual European Symposium. Springer 2832:2-5 (2003). cited by applicant
Yang, Yin et al. Authenticated join processing in outsourced databases. Proceedings of the 2009 ACM SIGMOD International Conference on Management of data. ACM:5-18 (2009). cited by applicant
Zhang, Yupeng et al. IntegriDB: Verifiable SQL for outsourced databases. Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security. ACM:1480-1491 (2015). cited by applicant
Zhang, Yupeng et al. vSQL: Verifying arbitrary SQL queries over dynamic outsourced databases. 2017 IEEE Symposium on Security and Privacy (SP). IEEE:863-880 (2017). cited by applicant
PCT/US2024/050368 International Search Report and Written Opinion dated Dec. 30, 2024. cited by applicant

---

Primary Examiner: Kim; Jung W

Assistant Examiner: Kong; Alan L

Attorney, Agent or Firm: WILSON SONSINI GOODRICH & ROSATI

Background/Summary

CROSS-REFERENCE (1) This application is the by-pass continuation of International Application No. PCT/US2024/050368, filed Oct. 28, 2024, which claims the benefit of Greek patent application No. 20240100620 filed Sep. 9, 2024, each of which are incorporated herein by reference in their entirety for all purposes.

BACKGROUND
(1) Smart contracts often times need to perform computation on-chain, for example, in order to compute the total amount of an asset owned by a specific address, or to compute the average price of a token during a certain period of time. Such computations share one common feature: They do not have access to the data directly—instead, the contracts have access to a commitment/digest d—of the data, exposed through a type of Merkle tree called Merkle Patricia Tries (MPTs). However, decommitting (i.e., recomputing the commitment) as well as performing the computation on-chain is particularly expensive.
(2) In the field of verifying queries, simple data structure queries (e.g., membership and non-membership) can be verified against digests of data structures. The database community considered the notion of verifiable JOINS, but the proof sizes can become large. Small proof sizes were achieved for a very small subset of SQL with IntegriDB. A more complete set of SQL queries were considered in vSQ. Finally, recently ZKSQL was proposed as a system for zero-knowledge SQL. However, ZKSQL is in a different model where there is interaction between the prover and the verifier.
SUMMARY
(3) The present disclosure addresses the above needs by providing a system that outputs SNARK proofs for arbitrary SQL queries on Merkle-type commitments of relational tables with reduced

proof size. The system may implement architectures such as zk-rollups, where smart contracts verify computational proofs of correctness (e.g., recursive succinct non-interactive arguments of knowledge (SNARK) proofs) for the desired computation on the commitment/digest d. The SNARK proof can be computed off-chain, significantly alleviating the gas cost required by smart contracts which will only have to read the public statement (i.e., Merkle Patricia commitment/digest d) and verify the respective SNARK proof π.

(4) Systems herein may implement the architectures in full generality: SQL language is the computation and relational tables are the "state" or "data." The present disclosure builds a system (referred to as PERSEUS) that outputs SNARK proofs for arbitrary SQL queries on Merkle-type commitments of relational tables.

(5) In an aspect, a computer-implemented method is provided for cryptographically verifying SQL queries. The method comprises: computing a succinct digest for a table stored in a database: receiving a query in structured query language (SQL) for querying data in the table; decomposing the query into one or more basic queries; and computing one proof per basic query in parallel and in a distributed fashion; and verifying the query by verifying the previously computed proofs in parallel, and checking for consistency of their public statements, according to the previously executed query decomposition.

(6) In a related yet separate aspect, a computer-implemented system is provided. The system comprises at least one processor and instructions executable by the at least one processor to cause the at least one processor to perform a method for cryptographically verifying SQL queries by performing operations comprising: computing a succinct digest for a table stored in a database: receiving a query in structured query language (SQL) for querying data in the table; decomposing the query into one or more basic queries; and computing one proof per basic query in parallel and in a distributed fashion; and verifying the query by verifying the previously computed proofs in parallel, and checking for consistency of their public statements, according to the previously executed query decomposition.

(7) In a related yet separate aspect, one or more non-transitory computer-readable storage media encoded with instructions executable by one or more processors is provided. The instructions are executable by the one or more processors to provide an application for cryptographically verifying SQL queries. The application comprises a software module computing (or configured to compute) a succinct digest for a table stored in a database: a software module receiving (or configured to receive) a query in structured query language (SQL) for querying data in the table; a software module decomposing (or configured to decompose) the query into one or more basic queries; a software module computing (or configured to compute), in parallel and in a distributed fashion, one proof per basic query; and a software module verifying (or configured to verify) the query by performing at least: verifying the previously computed proofs in parallel, and checking for consistency of public statements, according to the previously executed query decomposition.

(8) In some embodiments, the succinct digest of the table is collision resistant representation of the table. In some cases, the succinct digest of the table is computed using a Merkle tree. In some cases, the succinct digest of the table is computed using an order-agnostic hash function.

(9) In some embodiments, the table is a relational table and the database is a relational database. In some embodiments, each of the one or more basic queries comprises a verifiable SQL operator from a library of verifiable SQL operators. In some cases, a verifiable SQL operator is a SNARK proof for a statement involving a succinct digest of a table. In some instances, verifying the query comprises verifying, for each basic query, an input and output to the statement is consistent.

(10) In some embodiments, a size of a proof for verifying the query is dependent on a number of the one or more basic queries. In some embodiments, a size of a proof for verifying the query is not dependent on a size of the table. In some embodiments, computing and verifying the one or more proofs for the one or more basic queries comprise using Map/Reduce circuits for Reckle trees.

(11) Additional aspects and advantages of the present disclosure will become readily apparent to

those skilled in this art from the following detailed description, wherein only illustrative embodiments of the present disclosure are shown and described. As will be realized, the present disclosure is capable of other and different embodiments, and its several details are capable of modifications in various obvious respects, all without departing from the disclosure. Accordingly, the drawings and description are to be regarded as illustrative in nature, and not as restrictive.

INCORPORATION BY REFERENCE

(12) All publications, patents, and patent applications mentioned in this specification are herein incorporated by reference to the same extent as if each individual publication, patent, or patent application was specifically and individually indicated to be incorporated by reference. To the extent publications and patents or patent applications incorporated by reference contradict the disclosure contained in the specification, the specification is intended to supersede and/or take precedence over any such contradictory material.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) The novel features of the present disclosure are set forth with particularity in the appended claims. A better understanding of the features and advantages of the present disclosure will be obtained by reference to the following detailed description that sets forth illustrative embodiments, and the accompanying drawings (also “Figure” and “FIG.” herein), of which:

(2) FIG. **1** shows an example of Merkle digest and order-agnostic digest of a table.

(3) FIG. **2** shows an example of a library of verifiable operators (or gadgets) as shown in Table 1.

(4) FIG. **3** shows an example of computing hash C.sub.ν as a function of C.sub.L and C.sub.R in the general case when the number of leaves is not a power of two.

(5) FIG. **4** shows an example of computing hash C.sub.ν as a function of of C.sub.L and C.sub.R in the general case when the number of leaves is not a power of two.

(6) FIG. **5** shows an example of Map/Reduce circuits for Reckle trees, with vki indicating the verification key for circuit M.sub.i.

(7) FIG. **6** shows an example of merklization of the example table for Verifiable RANGE, with the data for each subtree stored in each internal node.

(8) FIG. **7** shows an example of the GROUPBY protocol as a MAP-REDUCE computation.

(9) FIG. **8** shows query Q**4** from the TPC-H benchmark.

(10) FIG. **9** shows query Q**5** from the TPC-H benchmark.

(11) FIG. **10** shows query Q**3** from the TPC-H benchmark.

DETAILED DESCRIPTION

(12) While various embodiments of the invention have been shown and described herein, it will be obvious to those skilled in the art that such embodiments are provided by way of example only. Numerous variations, changes, and substitutions may occur to those skilled in the art without departing from the invention. It should be understood that various alternatives to the embodiments of the invention described herein may be employed.

Certain Definitions

(13) Unless otherwise defined, all technical terms used herein have the same meaning as commonly understood by one of ordinary skills in the art to which this invention belongs.

(14) Reference throughout this specification to “some embodiments,” or “an embodiment,” means that a particular feature, structure, or characteristic described in connection with the embodiment is included in at least one embodiment. Thus, the appearances of the phrase “in some embodiment,” or “in an embodiment,” in various places throughout this specification are not necessarily all referring to the same embodiment. Furthermore, the particular features, structures, or characteristics may be combined in any suitable manner in one or more embodiments.

(15) As utilized herein, terms “component,” “system,” “interface,” “unit,” “module” and the like are intended to refer to a computer-related entity, hardware, software (e.g., in execution), and/or firmware. For example, a component can be a processor, a process running on a processor, an object, an executable, a program, a storage device, and/or a computer. By way of illustration, an application running on a server and the server can be a component. One or more components can reside within a process, and a component can be localized on one computer and/or distributed between two or more computers.
(16) Further, these components can execute from various computer readable media having various data structures stored thereon. The components can communicate via local and/or remote processes such as in accordance with a signal having one or more data packets (e.g., data from one component interacting with another component in a local system, distributed system, and/or across a network, e.g., the Internet, a local area network, a wide area network, etc. with other systems via the signal).
(17) As another example, a component can be an apparatus with specific functionality provided by mechanical parts operated by electric or electronic circuitry; the electric or electronic circuitry can be operated by a software application or a firmware application executed by one or more processors; the one or more processors can be internal or external to the apparatus and can execute at least a part of the software or firmware application. As yet another example, a component can be an apparatus that provides specific functionality through electronic components without mechanical parts; the electronic components can include one or more processors therein to execute software and/or firmware that confer(s), at least in part, the functionality of the electronic components. In some cases, a component can emulate an electronic component via a virtual machine, e.g., within a cloud computing system.
(18) Whenever the term “at least,” “greater than,” or “greater than or equal to” precedes the first numerical value in a series of two or more numerical values, the term “at least,” “greater than” or “greater than or equal to” applies to each of the numerical values in that series of numerical values. For example, greater than or equal to 1, 2, or 3 is equivalent to greater than or equal to 1, greater than or equal to 2, or greater than or equal to 3.
(19) Whenever the term “no more than,” “less than,” or “less than or equal to” precedes the first numerical value in a series of two or more numerical values, the term “no more than,” “less than,” or “less than or equal to” applies to each of the numerical values in that series of numerical values. For example, less than or equal to 3, 2, or 1 is equivalent to less than or equal to 3, less than or equal to 2, or less than or equal to 1.
(20) As used herein a processor encompasses one or more processors, for example a single processor, or a plurality of processors of a distributed processing system for example. A controller or processor as described herein generally comprises a tangible medium to store instructions to implement steps of a process, and the processor may comprise one or more of a central processing unit, programmable array logic, gate array logic, or a field programmable gate array, for example. In some cases, the one or more processors may be a programmable processor (e.g., a central processing unit (CPU) or a microcontroller), digital signal processors (DSPs), a field programmable gate array (FPGA) and/or one or more Advanced RISC Machine (ARM) processors. In some cases, the one or more processors may be operatively coupled to a non-transitory computer readable medium. The non-transitory computer readable medium can store logic, code, and/or program instructions executable by the one or more processors unit for performing one or more steps. The non-transitory computer readable medium can include one or more memory units (e.g., removable media or external storage such as an SD card or random access memory (RAM)). One or more methods or operations disclosed herein can be implemented in hardware components or combinations of hardware and software such as, for example, ASICs, special purpose computers, or general purpose computers.
(21) Moreover, the word “exemplary” is used herein to mean serving as an example, instance, or

illustration. Any aspect or design described herein as “exemplary” is not necessarily to be construed as preferred or advantageous over other aspects or designs. Rather, use of the word exemplary is intended to present concepts in a concrete fashion. As used in this application, the term “or” is intended to mean an inclusive “or” rather than an exclusive “or.” That is, unless specified otherwise, or clear from context, “X employs A or B” is intended to mean any of the natural inclusive permutations. That is, if X employs A; X employs B; or X employs both A and B, then “X employs A or B” is satisfied under any of the foregoing instances. In addition, the articles “a” and “an” as used in this application and the appended claims should generally be construed to mean “one or more” unless specified otherwise or clear from context to be directed to a singular form.

(22) As mentioned above, simple data structure queries (e.g., membership and non-membership) could be verified against digests of data structures. The database community considered the notion of verifiable JOINS, but the proof sizes can become large. Small proof sizes were achieved for a very small subset of SQL with IntegriDB or there is interaction required between the prover and the verifier.

(23) The present disclosure provides a system (Perseus) for cryptographically verifying a query in structured query language (SQL) (i.e., SQL query). In the system herein, relational tables are represented with succinct digests, computed either with a Merkle tree or with an order-agnostic hash function. All SQL queries may be decomposed into a series of basic SQL operators from a well-defined set of SQL operators (e.g., 10-20 SQL operators), such as RANGE or SORT, for which the system provides custom MAP/REDUCE protocols based on recursive SNARKs. To compute a proof for an SQL query, the prover can compute proofs for the involved SQL operators in a distributed fashion (both across operators and within each operator), leading to prover time that is sublinear in the size of the tables involved-similarly the produced proofs are also sublinear in size. In addition, the system allows proofs to be efficiently updated and without running the prover from scratch, whenever the underlying tables change. The preliminary experimental estimates on the well-known TPC-H benchmark used by the database community show that the system outperforms the baseline by 139× and 6.11× on proof size and updates, respectively.

(24) Digests of relational tables. The data object in the system may comprise a digest, a collision-resistant representation of a relational table T. Digests are part of the public statement that the contract reads and may never have to be decommitted on-chain. The system may comprise two types of digests: A Merkle digest is computed using a standard Merkle tree and introduces structure—a Merkle digest is denoted with d, and an order-agnostic digest, denoted o, is computed with an order-agnostic hash function, such as MULTHASH. In some cases, a particular type of digest is selected from the different types of digests based on the data in the relational table. For example, it may be more practical to run certain SQL queries on ordered data (such as ranges) whereas sometimes it is more convenient to accumulate the result of an SQL query as digest of an unordered array and potentially “lift” it later to a Merkle digest with structure. In some cases, the different types of digests may be associated with certain metadata, such as the attribute names of the underlying table as well as whether the table is indexed, and if so, on which attribute.

(25) FIG. **1** shows an example **100** of Merkle digest **103** and order-agnostic digest **105** of a table **101**. For the Merkle digest **103** of the table of 6 rows, since 6 is not a power of two, a subtree of the full binary tree of 8 leaves is used. The six rows are always placed as leaves (not internal nodes) of the 8-leaf Merkle tree. For the order-agnostic digest **105**, it may not include the “index” column.

(26) Verifiable SQL operators. Given an SQL query q that is to verify on a set of relational tables represented by either Merkle or order-agnostic digests, the system decomposes q into a series of SQL operators, drawn from a library of SQL verifiable operators that it defines and constructs herein. A verifiable SQL operator may be considered as a SNARK proof for a well-defined public statement that involves table digests. FIG. **2** shows an example of a library of verifiable operators (or gadgets) as shown in Table 1 **200**. For example, a verifiable SQL operator is a SNARK proof

for the public statement RANGE.sub.x (d, o, l, r),
(27) meaning that o is the order-agnostic digest of those rows of a table with Merkle digest d whose x attribute fall within the range [l, r]. To verify an SQL query, the prover may need to verify all involved SQL operators, making sure inputs and outputs to their public statements are consistent. This beneficially improves the verification efficiency as the total size of the proof for a specific SQL query is dependent only on the number of SQL operators used, but not on the size of the tables involved. In some cases, the system may further compress the final proof by using an additional level of recursion.
(28) Constructing verifiable SQL operators. In some embodiments, verifiable SQL operators may be constructed using any SNARK: decommit the digest d within the circuit, and then express the SQL query as, for example, a R1CS set of constraints. However, there are two problems with such an approach. First, it is difficult to know ahead of time what the sizes of the involved tables are going to be—these depend on the nature of the query, the parameters of the query and the tables themselves. Second, using such an approach does not allow for natural parallelism of the prover—the prover can be parallelized only to the extent that the underlying SNARK can be parallelized. To address the problems, systems and methods herein may use Reckle trees, a framework that allows one to produce up-datable proofs for MAP-REDUCE computations on data committed to by a Merkle tree in a massively parallel fashion and by combining, via recursive SNARKs, a series of constant-size circuits. The proofs are updatable that whenever an entry of the SQL table changes, the proofs can be updated in logarithmic time, without having to run the prover from scratch. For example, every operator of the library (e.g., Table 1 of FIG. **2**) can be expressed as a MAP-REDUCE computation and then provide an implementation using the Reckle trees framework. In addition, Reckle trees are extended to handle Merkle trees of arbitrary size, not just powers of two (see FIG. **1**), which beneficially allows for a practical implementation. Details about the Reckle tree framework and Map/Reduce proofs with RECKLE+ TREES can be the same as those described in patent application Ser. No. 18/661,204 filed on May 10, 2024 which is entirely incorporated herein by reference. For example, the Map/Reduce proofs with RECKLE+ TREES are described as below.
(29) Map/Reduce proofs with RECKLE+ TREES. Merkle trees may not support updatable verifiable computation over Merkle leaves. Merkle trees can only be used to prove memory content, but no computation over it. However, in certain applications (e.g., smart contract), it is desirable to compute an updatable proof for some arbitrary computation over the Merkle leaves, e.g., counting the number of Merkle leaves o satisfying an arbitrary function f(v)=1. For example, smart contracts can benefit by accessing historic chain MPT data to compute useful functions such as price volatility or BLS aggregate keys. Instead, such applications are currently enabled by the use of blockchain “oracles” that have to be blindly trusted to expose the correct output to the smart contract. In some embodiments, the present disclosure provides RECKLE+ TREES, an extension of RECKLE+ TREES that support updatable verifiable computation over Merkle leaves. With RECKLE+ TREES, a prover can commit to a memory M, and provide a proof of correctness for Map/Reduce computation on any subset of M.
(30) In some embodiments, the RECKLE+ TREES may be technically achieved by encoding, in the recursive circuit, not only the computation of the canonical bash and the Merkle hash (as in the case of batch proofs), but also the logic of the Map and the Reduce functions. The final Map/Reduce proof can be easily updated whenever the subset changes, without having to recompute it from scratch.
(31) In some cases, the methods herein may comprise embedding a computation of the batch hash inside the recursive Merkle verification via a hash-based accumulator (i.e., canonical hashing). The unique embedding beneficially allows for batch proofs being updated in logarithmic time, whenever a Merkle leaf (belonging to the batch or not) changes, by maintaining a data structure that stores previously-computed recursive proofs. In the cases of parallel computation, the batch

proofs are also computable in O(log n) parallel time-independent of the size of the batch.
(32) In alternative embodiments, an extension of Reckle trees or Reckle+ trees are provided. Reckle+ trees provide updatable and succinct proofs for certain types of Map/Reduce computations. For instance, a prover can commit to a memory M and produce a succinct proof for a Map/Reduce computation over a subset of M. The proof can be efficiently updated whenever or M changes.
(33) Whether succinct Merkle batch proofs can be built that are efficiently updatable relates to the notion of updatable SNARKs, that are disclosed herein. An updatable SNARK is a SNARK that is equipped with an additional algorithm $\pi' \leftarrow$ Update $((x, w), (x', w'), \pi)$. Where the update function takes as input a true public statement x along with its witness w (prover's knowledge) and its verifying proof π as well as an updated true public statement x' along with the updated witness w'. It outputs a verifying proof π' for x' without running the prover algorithm from scratch and ideally in time proportional to the distance (for some definition of distance) of (x, w) and (x', w').
(34) Reckle Trees
(35) In some embodiments of the present disclosure, a Reckle tree is provided. The Reckle tree may be a unique vector commitment scheme that supports updatable and succinct batch proofs using RECursive SNARKs and MerKLE trees. The term “vector commitment” as utilized herein generally refers to a cryptographic abstraction for “verifiable storage” and which can be implemented, for example, by Reckle trees or Merkle trees.
(36) A recursive SNARK is a SNARK that can call its verification algorithm within its circuit. Some of the SNARKs may be optimized for recursion, e.g., via the use of special curves. The framework disclosed herein may be compatible with any recursive SNARK (e.g., Plonky2).
(37) Reckle tree may work under a fully-dynamic setting for batch proofs: Assume a Reckle batch proof π.sub.l for a subset l of memory slots has been computed. Reckle tree can support the following updates to π.sub.l in logarithmic time: (a) change the value of element M [i], where $i \in I$; (b) change the value of element M [j], where j.Math.I; (c) extend the index set I to $I \cup w$ so that M[w] is also part of the batch; (d) remove index w from I so that M[w] is not part of the batch; (e) remove or add an element from the memory altogether. In this case, Reckle tree can rebalance following the same rules of standard data structures such as red-black trees or AVL trees.
(38) In some cases, updating a batch proof π.sub.I in Reckle tree is achieved through a batch-specific data structure Λ.sub.I that stores recursively-computed SNARKs proofs. Reckle tree can be naturally parallelizable. Assuming enough parallelism, any number of updates $T>1$ can be performed in O(log n) parallel time. For instance, a massively-parallel Reckle trees implementation can achieve up to 270× speedup over a sequential implementation.
(39) Reckle tree may have particularly low memory requirements: While they can make statements about n leaves, their memory requirements (excluding the underlying linear-size Merkle tree) scale logarithmically with n. Additionally, Reckle trees have the flexibility of not being tied to a specific SNARK: If a faster recursive SNARK implementation is introduced in the future (e.g., folding schemes), Reckle trees can use the faster technology seamlessly.
(40) In some embodiments, the method may comprise: letting I be the set of Merkle leaf indices for which to compute the batch proof, starting from the leaves l.sub.1, . . . , l.sub.|I| that belong to the batch l, Reckle tree run the SNARK recursion on the respective Merkle paths p.sub.1, . . . , p.sub.|I|, merging the paths when common ancestors of the leaves in I are encountered. While the paths are being traversed, Reckle trees may verify that the elements in I belong to the Merkle tree as well as compute a “batch” hash for the elements in I, eventually making this batch hash part of the public statement.
(41) In some cases, the batch hash is computed via canonical hashing, a deterministic and secure way to represent any subset of |I| leaves succinctly. It should be noted that any number-theoretic accumulator (or even the elements in the batch themselves) can be utilized. In some cases, the method herein may select canonical hashing to avoid encoding algebraic statements within the

circuits. This beneficially ensures the circuits' size not depend on the size and topology of the batch. If the final recursive proof verifies, that means that the batch hash corresponds to a valid subset of elements in the tree, and can be recomputed using the actual claimed batch as an input. The Reckle tree herein may distinguish from the conventional Merkle tree construction that every node ν in a Reckle tree, in addition to storing a Merkle hash C.sub.ν, can also store a recursively-computed SNARK proof π.sub.ν, depending on whether any of ν's descendants belongs to the batch I in question or not (For nodes that have no descendants in the batch, there is no need to store a SNARK proof.) The approach or methods herein can be easily extended to unbalanced q-ary Merkle.

(42) Referring back to the verifying SQL queries systems and methods herein, a query decomposition method may be provided.

(43) Example query decomposition. The system may support a simple SQL query using SQL operators from the operators library (e.g., Table 1). As an example, assume a table A that has two attributes x and y and the goal is to return the rows of A such that 105≤y≤250, projected at x and sorted by x. This query can be written in SQL as select x from A where 105≤y≤250 orderby x

(44) Let π.sub.C be the proof for the public statement CREATE.sub.y(d, A).

(45) Note that π.sub.C simply contains A and verifying it just involves recomputing the digest from scratch. It can also have A as a witness, but typically the correctness of the table digest is an orthogonal problem. The method may choose to sort A by y during setup because a range query on y is anticipated later. Then a range search proof π.sub.R on d is produced by using the SNARK for the statement RANGE.sub.y(d, o, 105,250),

(46) outputting an order-agnostic digest o. Next, the method produces the final single proof π.sub.S for the statement PROJECT-SORT.sub.x(o, d′).

(47) For this query, it will need to verify two proofs ITR for the statement (d, o, 105, 250) and π.sub.S for the statement (o, d′). The digests d, o and d′ and the underlying tables can be computed before starting to compute the proofs, and therefore the two proofs can be computed completely in parallel. Such parallelism capability beneficially improves the computational efficiency and reducing the computational time of the verification process. The parallelism for the system (PERSEUS) can be further improved by computing the individual proofs in parallel due to Reckle trees.

(48) Evaluation. The preliminary evaluation on the well-known TPC-H benchmark show that the proof size is approximately 139×(1.25 vs 174.52 MiB) and the updates are around 6.11×(14.56 vs 89.17 seconds) better than the baseline. The unoptimized prover is 4.19× slower than the baseline, which can be improved by implementing optimizations.

(49) Preliminaries

(50) Below are notation and definitions for SNARKs and Merkle Trees. Let λ be the security parameter and H: {0, 1}*.fwdarw.{0, 1}.sup.2λ denote a collision-resistant hash function. Let [n]=[0, n)={0,1, . . . ,n−1}.

(51) Succinct Non-Interactive Arguments of Knowledge [2]. Let  custom character be an efficiently computable binary relation that consists of pairs of the form (x, w), where x is a statement and w is a witness.

(52) Definition 2.1. A SNARK is a triple of PPT algorithms Π=(Setup, Prove, Verify) defined as follows:

(53) Setup (1.sup.λ,  custom character).fwdarw.(pk, vk): On input security parameter λ and the binary relation R, it outputs a common reference string consisting of the prover key and the verifier key (pk, vk).

(54) Prove (pk, x, w).fwdarw.π: On input pk, a statement x and the witness w, it outputs a proof π.

(55) Verify (vk, x, π).fwdarw.1/0: On input vk, a statement x, and a proof π, it outputs either 1 indicating accepting the statement or 0 for rejecting it.

(56) It also satisfies the following properties:

(57) Completeness: For all (x, w)∈custom character, the following holds:

(58) Pr(Verify(vk, x,) = 1 .Math. $\begin{matrix}(pk, \text{vk}) \leftarrow \text{Setup}(1^{\quad}, \mathcal{R}) \\ \leftarrow \text{prove}(pk, x, w)\end{matrix}$) = 1

(59) Knowledge Soundness: For any PPT adversary custom character, there exists a PPT extractor χcustom character such that the following probability is negligible in λ:

(60) Pr($\begin{matrix}\text{Verify}(\text{vk}, x, \quad) = 1 \\ \wedge\ \mathcal{R}(x, w) = 0\end{matrix}$ .Math. $\begin{matrix}(pk, \text{vk}) \leftarrow \text{Setup}(1^{\quad}, \mathcal{R}) \\ ((x, \quad); \leftarrow \quad \text{.Math.} \quad (pk, \text{vk}))\end{matrix}$) .

(61) (The notation (x, π); w) ← custom character‖.sub.χcustom character(pk, vk) means the following: After the adversary custom character outputs (x, x), it can run the extractor χ custom character on the adversary's state to output w. If the adversary outputs a verifying proof, then it must know a satisfying witness that can be extracted by looking into the adversary's state.)

(62) Succinctness: For any x and w, the length of the proof π is given by |π|=poly(λ).Math.polylog (|x|+|w|).

(63) Merkle trees. Let M be a memory of n slots. A Merkle tree is an algorithm to compute a succinct, collision-resistant representation C of M (also called digest) so that one can provide a small proof for the correctness of any memory slot M [i], while at the same time being able to update C in logarithmic time whenever a slot changes. Assume the memory slot values and the output of the H function have size 2λ bits each. The Merkle tree on an n-sized memory M can be constructed as follows. The n items will be leaves in a full binary tree of N leaves, where N is the closest power of 2 to n. The Merkle tree will not contain all nodes of the full binary tree but only those that are on the paths from the n leaves to the root, and their siblings. As such, some siblings might have no children, in which case it calls such nodes “external.” For every node ν in the Merkle tree T, the Merkle hash of ν, C.sub.ν, is computed as in FIG. **3**. FIG. **3** shows an example of computing hash C.sub.ν as a function of C.sub.L and C.sub.R in the general case when the number of leaves is not a power of two. The digest of the Merkle tree is the Merkle hash of the root node. The proof for a leaf comprises bashes along the path from the leaf to the root. It can be verified by using hashes in the proof to recompute the root digest C.

(64) Canonical hashing. Canonical hashing is an algorithm to compute a digest of some subset I of the leaves of a Merkle tree. Define the canonical hash of a node ν of Merkle tree T (with respect to some subset I), denoted d(ν, I), recursively as in FIG. **4**. FIG. **4** shows an example of computing hash C.sub.ν as a function of of C.sub.L and C.sub.R in the general case when the number of leaves is not a power of two.

(65) Order-agnostic canonical hashing. Order-agnostic canonical hashing is the same algorithm with canonical hashing except that the hash function H is replaced with an order-agnostic hash function O. In particular O: custom character.sup.2.fwdarw.custom character is a function that can be used to output a collision resistant hash of a set S={x.sub.1 . . . , x.sub.n} of elements in custom character, in the following way: For i=1, . . . , n, set h.sub.i=O(h.sub.i-1, x.sub.i), where h.sub.0 is a fixed constant. Importantly, the output hash does not depend on the order in which the elements of the set are being considered. For the implementation, the system uses the order agnostic hash.

(66) Reckle trees. Reckle trees is a way to compute updatable proofs for Map/Reduce computations in a massively parallel fashion as described elsewhere herein. In particular, let custom character and custom character denote the domain of the input and output of the computation, respectively. Reckle trees work with the following abstraction, in terms of predicates, for Map/Reduce: Map: custom character×custom character.fwdarw.0/1; Reduce: custom character× custom character×custom character.fwdarw.0/1.

(67) In particular, Map is a predicate that takes as input both the input to the MAP computation and the output of the MAP computation and returns 1 if and only if the input and output are consistent.

Define the Reduce predicate similarly.
(68) Assume to execute the Map/Reduce computation on a subset I.Math.[n] of the memory slots which have (d, o) as their canonical hash and order-agnostic hash. With Reckle trees a proof can be computed for the following public statement: MR(out, C, d, o),
(69) out is the output of the Map/Reduce computation on some set of leaves I of a Merkle tree whose Merkle root is C and whose root has (d, o) as canonical hash and order-agnostic canonical hash with respect to I.
(70) In some cases, computing this proof comprises using recursive SNARKs—the circuits in FIG. **5**. FIG. **5** shows an example of Map/Reduce circuits for Reckle trees, with vki indicating the verification key for circuit M.sub.i.
(71) In particular, the first thing that the prover needs to do is to compute the witnesses that are required to run the SNARK proof. To do that, the prover performs the following: 1) The prover computes the tree T.sub.I formed by Merkle proofs {π.sub.i}.sub.i∈I. For each node ν of T.sub.I it stores the respective value C.sub.ν that that can be found in or computed from the proofs {π.sub.i}.sub.i∈I; 2) Let now T.sub.I′ ⊂T.sub.I be the subtree of T.sub.I that contains just the nodes on the paths from I to the root. The prover computes the canonical hash d(ν, I) for all nodes ν∈T.sub.I′ with respect to I as well as the output of the Map/Reduce computation.
(72) After the witness is computed, the method proceeds with computing the recursive SNARK proofs that eventually will output the Map proof. The method first produces the public parameters by running the setup of the SNARK (pk.sub.i, vk.sub.i)←Setup (1.sup.λ, M.sub.i) for the circuit M.sub.i, where i=0, . . . custom character−1. Consider now the set of indices I for which it is calculating the Map/Reduce proof and let T.sub.I′ be the subtree as defined above. Let V.sub.I be the nodes of T.sub.I′ at level l=1, . . . custom character. To compute the Map/Reduce proof, it follows the procedure below.
(73) For all levels l=0, . . . custom character−1, for all nodes ν∈V.sub.ls
(74) Let L be ν's left child and R be ν's right child in T.sub.l;
(75) Set

$$\pi.\mathrm{sub}.\nu=\mathrm{Prove}(pk.\mathrm{sub}.i,x.\mathrm{sub}.\nu,(w.\mathrm{sub}.L,w.\mathrm{sub}.R)), \quad (1)$$

where

$$x.\mathrm{sub}.\nu=(C.\mathrm{sub}.\nu,d.\mathrm{sub}.\nu,o.\mathrm{sub}.\nu,\mathrm{out}.\mathrm{sub}.\nu),$$

and

$$w.\mathrm{sub}.L=(C.\mathrm{sub}.L,d.\mathrm{sub}.L,o.\mathrm{sub}.L,\mathrm{out}.\mathrm{sub}.L,\pi.\mathrm{sub}.L),$$

and

$$w.\mathrm{sub}.R=(C.\mathrm{sub}.R,d.\mathrm{sub}.R,o.\mathrm{sub}.R,\mathrm{out}.\mathrm{sub}.R,\pi.\mathrm{sub}.R).$$

(76) The final Map/Reduce proof for index set I will be πr, where r is the root of TI.
(77) Verifiable SQL Operators
(78) Systems of the present disclosure implement the verifiable SQL operators from the library (e.g., Table 1) using the Map/Reduce framework. As an example, assume that every row ν of the relational tables have two attributes, key(ν) and value(ν), and assume that they are indexed by key(ν). This will be generalized to arbitrary number of attributes and arbitrary indices later.
(79) Verifiable CREATE Operator
(80) The method may compute the digest of a Table T using a Merkle tree. On input a table T, CREATE operator will compute d by prepending index(ν) to every row key(ν)‖value(ν) of T. Therefore to provide a proof for the public statement CREATE (d, T),
(81) meaning d is the Merkle digest of a table T whose rows are of the type index(ν)‖key(ν)‖value(ν).
(82) one has to output just T. Note that it can also choose to have T as witness in which case the method may use the Map/Reduce framework. In some cases, the method may not implement Map/Reduce framework when the digest d of the table where the SQL query will be executed is considered to be trusted and that it has been computed correctly.

(83) Verifiable SORT Operator
(84) Let d be the digest of a Merkle tree of an unsorted table (on key(v)) of n rows. Let now d′ be the digest of a Merkle tree on the same table T, but sorted on key(v), It will provide a SNARK construction for the following public statement: SORT (d, d′),
(85) d′ is the digest of a table that has the same elements with another table represented by digest d, but is rearranged to be sorted by the key of the table.
(86) For example, if d represents the 4-row table index
(87) TABLE-US-00001 index key value 0 3 7 1 1 15 2 13 22 3 5 104 then d′ represents the 4-row table index
(88) TABLE-US-00002 index key value 0 1 15 1 3 7 2 5 104 3 13 22
(89) The proof for the statement SORT (d, d′) consists of two proofs, the proof of order for (d′, o)—meaning that d′ is the digest of an ordered table with order-agnostic hash o, and the proof of completeness for (d, o)—meaning that o is the order-agnostic hash for a table whose digest is d. Clearly, if both proofs verify, it follows that d′ is the “sorted” digest of d.
(90) Proof of order. For the proof of order π.sub.o, let d′ be the digest of the sorted table. To check that d′ corresponds to a table that is sorted and that it has the appropriate indexing, is to define the MAP and REDUCE functions as follows. For MAP, the input is a leaf element index(v)‖key(v)‖value(v)
(91) and the output is four values: min=max=key(v) and minInd=maxInd=index(v). REDUCE will take as input eight values (min.sub.L, minInd.sub.L), (max.sub.L,maxInd.sub.L), (min.sub.R,minInd.sub.R), (max.sub.R,maxInd.sub.R) and will output (min, minInd) and (max, maxInd) if the following tests hold, (1) max.sub.L≤min.sub.R; (2) maxInd.sub.L=minInd.sub.R−1; (3) max=max.sub.R and maxInd=maxInd.sub.R; and (4) min=min.sub.R and minInd=minInd.sub.R. So a proof of order is a proof for the statement MR (d′, *, o, [(min,minInd), (max,maxind)]) for the MAP and REDUCE functions as described above. It indicated with * the third argument of the MR statement. This is because it is not necessary to consider all the elements in the table so it does not have to compute the canonical hash—just the order agnostic canonical hash is enough.
(92) Proof of completeness. To prove that o is the order-agnostic hash for a table whose digest is d, the method produces a proof for the public statement MR (d, d, o, ⊥). Use ⊥ to indicate that it is not using any MAP-REDUCE function. Importantly the second and third argument must both be equal to d, meaning that the order-agnostic hash o corresponds to the whole table represented by d.
(93) AGNOSTIC-SORT. In case the unsorted table is given as input as an order agnostic hash o, it only needs to perform a proof of completeness. In this case the primitive AGNOSTIC-SORT (o, d′) is called.
(94) Verifiable RANGE Operator
(95) Let now d be the digest of a table that is sorted on key(v). The method provides a SNARK construction for the public statement RANGE(d, o, a, b), is the order-agnostic hash of a table that contains all rows with key in [a, b] from a table represented by digest d.
(96) For example if d represents the 4-row table
(97) TABLE-US-00003 index key value 0 1 15 1 3 7 2 5 104 3 13 22 and a=2 and b=7 then o will represent the 2-row table
(98) TABLE-US-00004 key value 13 7 7 104
(99) The reason to not include the index column is because the order-agnostic hash does not include index. The method will implement the verifiable RANGE functionality using the MAP-REDUCE framework. In particular, the MAP function applied on a leaf index(v)‖key(v)‖value(v) and on the variables sumOut, sumIn, o, maxInd, minInd returns 1 if maxInd=minInd=index(v), for some t.
(100) sumIn=1 and sumOut=0 and o=key(v)‖value(v) if key(v)∈[a, b]
(101) sumIn=0 and sumOut=1 and o=1 if key(v).Math.[a, b].

(102) The REDUCE function will take input sumIn.sub.L, sumOut.sub.L, o.sub.L, t.sub.L, minInd.sub.L, maxInd.sub.L; sumIn.sub.R, sumOut.sub.R, o.sub.R, t.sub.R, minInd.sub.R, maxInd.sub.R sumIn, sumOut, o, minInd and maxInd and outputs 1 iff sumin=sumin.sub.L+sumin.sub.R, sumOut=sumOut.sub.L+sumOut.sub.R, o=O (oL, oR), minind=min(minind.sub.L, minind.sub.R) and maxind=max(maxind.sub.L, maxind.sub.R).
(103) A proof for RANGE (d, d′, a, b) is a proof for the statement MR (d, *, *, (sumin, sumOut, o, minind, maxind)), where o is the order-agnostic hash of the range if sumOut=2 (boundary values have been considered) and maxind−minind=sumin+1 (nothing has been omitted that lies within the boundary values) and mini nd=−1 (the indices have been adjusted). To finally get to d′, it produces an additional proof for AGNOSTIC-SORT (o, d′).
(104) JOIN-RANGE. JOIN-RANGE (d, o, a, b, val) is the same with RANGE with the only difference that it takes as input an additional argument (val) and appends this value to the final output. In particular, for the previous example is it set val=37 the output table will be
(105) TABLE-US-00005 key value 3 37||7 5 37||104
(106) Note it is easy to adjust RANGE to JOIN-RANGE. All needs to do is pass the value val in the MAP function and make it part of the public statement, while at the same time checking that all leaves used the same value val.
(107) FIG. **6** shows an example of merklization of the example table for Verifiable RANGE, with the data for each subtree stored in each internal node. Each subtree tracks its maximum and minimum, the count of elements in the range, the count of elements not in the range, and the order agnostic hash of rows whose key is in the range Additionally, each node stores a recursive SNARK proof that it was correctly constructed and that the proofs in the child nodes are correct.
(108) Verifiable EQUIJOIN Operator
(109) Let d be the Merkle digest of a table T and d′ be the Merkle digest of a table T′. Let also D the Merkle digest of the EQUIJOIN of the tables represented by d and d′. Construct a SNARK for the public statement EQUIJOIN (d, d′, o). D is the is the digest of the table representing the EQUIJOIN of the tables represented by Merkle digests d and d′.
(110) For example, if d represents the table
(111) TABLE-US-00006 Index key value 0 1 15 1 3 7 2 5 104 3 13 22 and d′ represents the table
(112) TABLE-US-00007 Index key value 0 1 22 1 1 71 2 5 11 3 5 57 D should represent the table
(113) TABLE-US-00008 Index key value 0 1 15||23 1 1 15||71 2 5 104||11 3 5 104||57
(114) It will produce an EQUIJOIN proof in two steps. In the first step, it will produce an order-agnostic hash of the rows in the EQUIJOIN, i.e., an order-agnostic hash of the elements (1, 15‖23), (1, 15‖71), (5, 104‖71), (5, 104‖57).
(115) Then it will use AGNOSTIC-SORT to map the above order-agnostic hash to a Merkle digest representing the table above.
(116) Therefore the rest of this section will focus on how to produce the order-agnostic hash of the elements in the EQUIJOIN. The method uses the MAP-REDUCE framework on the first table (represented by d) as follows. The MAP function on leaf v takes as input index(v)‖k=key(v)‖v=value(v)
(117) A proof Irk for the public statement JOIN-RANGE (d′, o, k, k, v), and output 1 iff the proof Irk verifies and the arguments in JOIN-RANGE are consistent with k and v of the node in question. As for the REDUCE function, it takes as input the order agnostic hashes o.sub.L and o.sub.R as well as the digests of the second table d′.sub.L and d′.sub.R and outputs o and d′.sub.o.sup.ut and checks whether o=o.sub.L=o.sub.R and d′.sub.L=d′.sub.R=do.sup.ut=d′. Note that d′ is a constant, the Merkle digest of the second table. The reason to output d′ in the REDUCE function is to ensure that every leaf node is running JOIN-RANGE on the same table d.
(118) In the end, the public statement it provides a proof for is MR (d, d, *, (o, d′)).
(119) Note that the first two arguments of the MR statement above must be equal to d, indicating that it runs the MAP function on every single leaf of the first table, as is required in order to have a

complete JOIN.
(120) Reducing the number of JOIN-RANGE proofs. In the above construction of EQUIJOIN, it is required to compute a JOIN-RANGE proof for all leafs of the first table that is joining. However, in some cases most of the leaves are not contained in the second table and therefore no rows would be contributed to the result of the EQUIJOIN by considering those leaves. The goal here is to minimize the number of JOIN-RANGE proofs to compute. For example, imagine the first table is has the following keys on the JOIN attribute (1, 2, 3, 4, 5, 6, 7, 8, 9) while the second table has the following keys on the join attribute (1, 1, 1, 1, 1, 9, 9, 9, 9, 9)
(121) Instead of doing 9 JOIN-RANGE proofs, it can do just 3. One for 1, one for the interval [2,8] (which will return an empty set) and one for 9.
(122) Verifiable GROUPBYAGG
(123) Let d be the digest of the input table and and let o be the order agnostic hash of the table produced by running GROUPBY with aggregate function f. For example, if d is the digest of the table
(124) TABLE-US-00009 Index key value 0 1 15 1 1 7 2 5 104 3 5 22 and the aggregate function is sum then o should be the order-agnostic hash of the table
(125) TABLE-US-00010 key value 1 22 5 126
(126) The method shows how to build a SNARK for the public statement GROUPBYAGG(d, o). FIG. **7** shows an example of the GROUPBY protocol as a MAP-REDUCE computation. Internal nodes track the leftmost key in the subtree, aggregate of values in the subtree for that key, the rightmost key and its aggregate in the subtree, and an order-agnostic hash of the key, aggregate pairs for keys strictly between the left and rightmost keys.
(127) Again, using the MAP-REDUCE framework as follows: Since the table is sorted, every node v of the tree covers a leaf region that can be divided in three areas: The left area contains keys that might spread to other subtrees to the left of v. The right area contains keys that might spread to other subtrees to the right of v. The middle area contains all keys that cannot exist in other subtrees and that can be safely accumulated as part of the final result. The method makes sure that every node of the tree must keep track of at most three values: The left running aggregate LA, the right running aggregate RA and the running accumulation of aggregates ACC. Below shows the MAP function in detail. The MAP function will take as input index (v)‖key(v)‖value(v) as well as: (k.sub.la, LA), the left aggregate; (k.sub.ra, RA), the right aggregate; ACC, the accumulation of aggregates. and will output 1 iff LA=RA=value(v), kla=kra=key(v) an ACC=1
(128) The REDUCE function will work as follows. It merges two inputs. (k.sub.la.sup.(L) LA.sub.L, k.sub.ra.sup.(L) RA.sub.L, ACC.sub.L (k.sub.la.sup.(R) LA.sub.R, k.sub.ra.sup.(R) RA.sub.R, ACC.sub.R into one, (K.sub.la,LA), (K.sub.ra,RA), ACC according to the following cases:
(129) If k.sub.la.sup.(L)=k.sub.la.sup.(R) it checks that ACC=ACC.sub.L*ACC.sub.R and that k.sub.la=k.sub.ra=k and that LA=RA=f(LA.sub.L, LA.sub.R, RA.sub.L, RA.sub.R).
(130) If k.sub.la.sup.(L)<k.sub.la.sup.(R) raand neither k.sub.la.sup.(R) nor k.sub.la.sup.(L) lie strictly between them, merge pairs with the same key to a single (key, Agg) pair with the aggregate being the application of the function f to the respective values. In this case, there will be two such pairs and then it checks that ACC=ACC.sub.L*ACC.sub.R and it checks that kla=k.sub.la.sup.(L), k.sub.ra=k.sub.la.sup.(R) and that L.sub.A=AGG.sub.L (the aggregate with key k.sub.la) and RA=AGG.sub.R (the aggregate with key k.sub.ra).
(131) If k.sub.la.sup.(L)<k.sub.la.sup.(R) and k.sub.la.sup.(R) or k.sub.la.sup.(R)alie strictly between them, merge pairs with matching keys. In the case of one “interior” (key, Agg) pair, set ACCC=0 (key, Agg); in the case of two pairs, set ACCC=0 (key.sub.C1, Agg.sub.C1)*0 (key.sub.C2, Agg.sub.C2). Then check that ACC=ACC.sub.L*ACC.sub.R*ACCC and it checks that k.sub.la=k.sub.la.sup.(L), k.sub.ra=k.sub.la.sup.(R) raand that LA=AGG.sub.L and RA=AGG.sub.R.

(132) Verifiable SELECT Operator

(133) One of the most important primitives is selection. In particular the public statement to prove is parameterized by the digest of the input table d, the order-agnostic hash of the output table o, and the predicate that apply to all leaves of the table, expressed with a function pred—i.e., SELECT(d, o, pred). To build a SNARK for selection the method uses the Map/Reduce framework. For the MAP predicate, out is either x (the value passes the filter) or the default value l. The REDUCE function will just multiply the results that it receives from the children nodes and will also ensure that the canonical hash matches the Merkle hash, i.e., that all nodes have been considered.

(134) Extending to Multiple Attributes

(135) The method and system herein may extend the verifiable SQL operators to multiple attributes. For instance, the method may view key(v)‖value(v) just as row(v) which is a bitstring of k.Math.m bits, where m is the fixed number of bits used to represent each attribute, and k is the number of attributes. Under this new definition the digest d output by CREATE(d, T) (where T is a table of n rows) will contain the information (n, k, m, att, ind) as “metadata,” where att is a list of attributes of the form: (1, att.sub.1), (2, att.sub.2), . . . (k, att.sub.k)

(136) and att.sub.i are strings describing the name of each attribute, such as “employeeName” or “orders.” Note that each attribute is represented by an integer in [l, k]. Without loss of generality, assume that this integer is the same across tables. In implementation, two integers are provided, when for example, to join two tables on the same attribute. Finally, note that the final piece of metadata is ind, which indicates whether the table is indexed on attribute ind. If not, then ind is set to null. Rewrite the public statement that it can prove for multiple attributes.

(137) CREATE.sub.x (d, 7), where d contains as metadata (n, m, k, att, ind), as defined above. Use x to indicate an attribute for the table that is used as index during creation (i.e., the rows of the table will be sorted based on x).

(138) SELECT.sub.x (d, o, func), where x is the attribute of d on which the selection is performed.

(139) SORT.sub.x (d, d′), where x is the attribute of d's att on which the sorting is performed. Note that the in d metadata field of d′ must be set to x.

(140) AGNOSTIC-SORT.sub.x (o, d′). Same as above. Note that agnostic sort digest o also contains metadata, but the in d field is by definition null.

(141) RANGE.sub.x (d, o, l, r), where x is the attribute on which the range is performed. Note d should have the in d argument set to x, i.e., it has to be sorted.

(142) JOIN-RANGE.sub.x (d, o, l, r, val), as above with the difference that o's attribute list will be updated to contain the attributes that val contains.

(143) EQUIJOIN.sub.x (d, d′, o), where x is the attribute where the JOIN is computed on. Note that the metadata of o are updated to include a concatenation of d's and d′'s attributes, with the difference that attribute x is copied only once.

(144) GROUPBYAGG.sub.x,y (d, o, func), where x is the attribute on which it performs the groupby and y is the attribute it applies the function on. Note that d must be sorted.

(145) Verifiable PROJECT Operator

(146) Let d be the digest of the table to compute a project on. To prove the public statement PROJECT, (d, o), where x is a list of attributes that is to keep, it is to see how the MAP-REDUCE framework can be used to implement project. For the MAP function, it will just include the arguments that indicated by the attribute list, and the REDUCE function will perform a simple multiplication, as shown in SELECT.

(147) Verifiable ADD Operator

(148) Let d be the digest of the table add an attribute x. Again, it can perform as simple MAP-REDUCE computation and produce a SNARK proof for the public statement ADD.sub.x (d, d′). Without loss of generality it can assume that the value of the new attribute x is set to a default predetermined value.

(149) Query Decomposition from Verifiable SQL Operators

(150) It shows precisely how to produce proof for the query Q**4** derived from the TPC-H benchmark. FIG. **8** shows query Q**4** from the TPC-H benchmark. FIG. **9** shows query Q**5** from the TPC-H benchmark. As shown below, in query Q**4** there are exactly two tables involved, lineitem and orders. Let lin and ord be the digests created for these tables respectively, by calling CREATE.sub.orderdate(ord, orders) and CREATE.sub.orderkey(lin, lineitem).
(151) The first thing to do is to run a range search on ord by calling RANGE.sub.orderdate(ord, o.sub.1, [DATE], [DATE]+3).
(152) Then it does a selection query on lin by calling SELECT.sub.commitdate,receiptdate(lin, o.sub.2, <), where < indicates the predicate is "less than." At this point it has produced verifiable digest o**1** that contains a subset of rows from orders such that the attribute orderdate falls within a certain range and a verifiable digest of o.sub.2 that contains a subset of rows from lineitem such that commitdate is less than receiptdate. It produces a lifted Merkle digest d.sub.1 for o.sub.1, via LIFT(o.sub.1, d.sub.1) and a sorted Merkle digest d.sub.2 for o.sub.2, via SORT-AGNOSTIC.sub.orderkey(o.sub.2, d.sub.2)
(153) Due to the existence of the SQL exists operator, it needs to further filter the rows of d**1** and keep only those whose orderkey appears at least once in the orderkey of d.sub.2. To do this final filtering it can run the select operation SELECT.sub.orderdate(d.sub.1, o.sub.3, ∈d.sub.2), where ∈d.sub.2 is the predicate "belongs in d.sub.2." To implement this predicate, it provides a range proof for the value of the respective orderdate attribute of d.sub.1 in d.sub.2 and the predicate checks whether the output order-agnostic hash is 1 (i.e., empty range) or not (in which case the row is kept). In particular for every orderdate value v it produces a proof for the statement RANGE.sub.orderdate(d.sub.2, o.sub.4, v, v) and the produced proof is being given as input to the ∈d.sub.2 predicate. At this point o.sub.4 is an order-agnostic hash of those rows of orders satisfying all predicates of Q**4**. The final step is to do a group-by. For that it produces an ordered version d.sub.4 of o.sub.4 by orderpriority by running SORT-AGNOSTIC.sub.orderpriority(o.sub.4, d.sub.4).
(154) Then it adds a "count" column by running ADD.sub.count(d.sub.4, d.sub.5) and finally it performs a "groupby" by running GROUPBYAGG.sub.orderpriority,count(d.sub.5, O, sumcount), where sumcount just counts the number of summands. Finally, it does a project on orderpriority and count by doing a LIFT(O, D) and then doing a PROJECT.sub.orderpriority,count(D, F)
(155) All in all, verifying this query requires the computation of 10 proofs. Note that all of these proofs can be done in parallel. First it shows that 6 tables are involved in this query and a digest is created for each one. CREATE.sub.orderkey(ord_ok, orders) CREATE.sub.suppkey(lin_sk,lineitem) CREATE.sub.nationkey(cus_nk, customer) CREATE.sub.nationkey(sup_nk, supplier) CREATE.sub.regionkey(nat_rk, nation) CREATE.sub.regionkey(reg_rk, region)
(156) Two tables can be reduced in size by selections: apply the user's choice of region to the reg_rk table and do a range search on ord_ok using the user's choice of date. SELECT.sub.name'[REGION]'(reg_rk, reg_rk_choice) RANGE.sub.orderdate(ord_ck, ord_ck_rng, '[DATE]', '[DATE]'+365)
(157) Now it can prepare for the joins. Each table is used in more than one join so it will need to create auxiliary digests for some tables which have been sorted by some other attribute. The first join it will do is to get the nations that are in the user's choice of region. EQUIJOIN.sub.regionkey(nat_rk, reg_rk_choice, do)
(158) Then it sorts this table by nationkey. SORT.sub.nationkey(d.sub.0, d.sub.1)
(159) Then join the suppliers table and then the customer table on nationkey: EQUIJOIN.sub.nationkey(sup_nk, d.sub.1, d.sub.2) EQUIJOIN.sub.nationkey(cus_nk, d.sub.2, d.sub.3)
(160) Now sort on suppkey so it can join the lineitem table. SORT.sub.suppkey(d.sub.3, d.sub.4) EQUIJOIN.sub.suppkey(lin_sk, d.sub.4, d.sub.5)

(161) Sort and join again, this time joining the order table on orderkey. SORT.sub.orderkey(d.sub.5, d.sub.6) EQUIJOIN.sub.orderkey(ord.sub.0k, d.sub.6, d.sub.7)
(162) One more round of sorting and joining provides the full table that are aggregating over. Because the six joins in the query form a cycle it creates a copy of the table that completes the cycle and project away all of its attributes except the one it joins over. This prevents duplicate attributes in the resulting table. CREATE.sub.custkey(cus_ck, customer) PROJECT.sub.custkey(cus_only_ck, cus.sub.ck) SORT.sub.custkey(d.sub.7,d.sub.8) EQUIJOIN.sub.custkey(cus_only_ck, d.sub.8, d.sub.9)
(163) Now that the full table has been generated it performs the aggregate sum. First it adds a column to hold the revenue, then populates it with a GROUPBYAGG. The function sumrevenue computes revenue as extendedprice*(1−discount) and sums the results. ADD.sub.revenue(d.sub.9, d.sub.10) GROUPBYAGG.sub.name,revenue(d.sub.10, d.sub.11, sumrevenue)
(164) Project away attributes no longer needed: PROJECT.sub.name,revenue(d.sub.11, d.sub.12)
(165) Finally sort by revenue: SORTrevenue(d.sub.12, res, >).
Evaluation
(166) The results of preliminary estimates of Perseus on query Q**3** from TPC-H benchmarks are shown and compare it with ZKSQL, the only verifiable DB protocol with publicly available source code.
(167) Set the database size to contain 60k rows in the lineitem table, corresponding to a scaling factor of 0.01 as defined in the TPC-H specifications. In the experiments, it devices the following query execution plan for the query Q**3** from TPC-H (see FIG. **10**). FIG. **10** shows query Q**3** from the TPC-H benchmark:
(168) Select the subset of rows from customer, orders, and lineitems using SELECT operator by applying respective predicates.
(169) Join the tables customers and orders using the EQUIJOIN operator, say the resultant table is denoted as customers_orders. Recall that EQUIJOIN requires the proof of JOIN-RANGE for each leaf in customer table.
(170) Sort the results of customers_orders based on the attribute o_orderkey.
(171) Similar to step 2, join the tables customers_orders and orders using the EQUIJOIN operator, say the resultant table is denoted as customers_orders_lineitem.
(172) Apply the GROUPBY operator on customers_orders_lineitem.
(173) Note that the proof computation in each step of Q**3** does not depend on the proofs from another step of the query plan. Thus, all the steps in Perseus can be executed concurrently.
(174) Experimental setup. The experiment used AWS c7i.8xlarge EC2 instances (32vCPU, 64 GiB memory) in all the experiments, including the baseline. To compute the execution times of the baseline approach, it used the artifacts provided with ZKSQL. Use Plonky2 proof system and write all our circuits in Rust. Let both the implementation and the baseline to use all the available parallelism provided by the framework.
(175) The experiments set the scaling factor, which is a parameter in TPC-H deter-mining the database sizes, to 0.01. Thus, the table customer contains 1500 rows, order contains 15,000 rows, and lineitem contains 60,175 rows. In Perseus, it padded the tables to next power of two with default values. The choice of parameters such as ‘[SEGMENT]’ and ‘[DATE]’, to run Q**3** is identical to the choice of parameters in the baseline.
(176) Implementation. The construction is amenable to distributed proving. Thus, the implementation uses the distributed systems architecture as described above for all the experiments. Moreover, in the experiments, it used a cluster of 200 machines.
(177) The experiments implemented the circuit of JOIN-RANGE operator in Plonky2. However, it reused the circuits from Reckle trees to simulate the cost of other operators used in Q**3** (FIG. **10**). For instance, to simulate verifiable EQUIJOIN circuit, it just needed to add the cost of combining the results of two reduce operations to the circuits from Reckle trees.

(178) Results. The findings regarding the performance of Perseus on Q**3** are as follows:
(179) Proof size. The total proof size is around 1.25 MiB without any proof aggregation technique to combine individual operator proofs. However, proof size in the baseline approach is 174.52 MIB. Thus representing an improvement of up to 139× than the baseline.
(180) Update time. ZKSQL requires recomputing all proofs whenever the underlying data changes. Thus it requires 89.17 seconds. The efficient tree structure of our database allows us to update a proof on a single update in 14.56 seconds. This represents a speed of 6.11× than the baseline.
(181) Prover time. The proving time is estimated to be 373.2 seconds. However, the baseline approach takes 89.17 seconds. This slowdown is a one-time cost as our prover can exploit fast updates whenever the underlying data changes rather than recomputing the proof from scratch.
(182) While preferred embodiments of the present subject matter have been shown and described herein, it will be obvious to those skilled in the art that such embodiments are provided by way of example only. Numerous variations, changes, and substitutions will now occur to those skilled in the art without departing from the present subject matter. It should be understood that various alternatives to the embodiments of the present subject matter described herein may be employed in practicing the present subject matter.

Claims

1. A computer-implemented method for cryptographically verifying SQL queries, the method comprising: a) computing a succinct digest for a table stored in a database; b) receiving a query in structured query language (SQL) for querying data in the table; c) decomposing the query into one or more basic queries, wherein each of the one or more basic queries comprises a verifiable SQL operator from a library of verifiable SQL operators, and wherein the verifiable SQL operator is a Succinct Non-interactive Argument of Knowledge (SNARK) proof for a statement involving a succinct digest of a table; d) computing one proof per basic query in parallel and in a distributed fashion; and e) verifying the query by verifying the previously computed proofs in parallel, and checking for consistency of their public statements, according to the previously executed query decomposition.
2. The method of claim 1, wherein the succinct digest of the table is collision resistant representation of the table.
3. The method of claim 2, wherein the succinct digest of the table is computed using a Merkle tree.
4. The method of claim 2, wherein the succinct digest of the table is computed using an order-agnostic hash function.
5. The method of claim 2, wherein the succinct digest encodes metadata related to an attribute name of the table or an indexing of the table.
6. The method of claim 1, wherein the table is a relational table and the database is a relational database.
7. The method of claim 1, wherein verifying the query comprises verifying, for each basic query, an input and output to the statement is consistent.
8. The method of claim 1, wherein a size of a proof for verifying the query is dependent on a number of the one or more basic queries.
9. The method of claim 1, wherein a size of a proof for verifying the query is not dependent on a size of the table.
10. The method of claim 1, wherein computing and verifying the one or more proofs for the one or more basic queries comprises using Map/Reduce circuits for Reckle trees.
11. A computer-implemented system comprising at least one processor and instructions executable by the at least one processor to cause the at least one processor to perform a method for cryptographically verifying SQL queries by performing operations comprising: a) computing a succinct digest for a table stored in a database; b) receiving a query in structured query language

(SQL) for querying data in the table; c) decomposing the query into one or more basic queries, wherein each of the one or more basic queries comprises a verifiable SQL operator from a library of verifiable SQL operators, and wherein the verifiable SQL operator is a Succinct Non-interactive Argument of Knowledge (SNARK) proof for a statement involving a succinct digest of a table; d) computing one proof per basic query in parallel and in a distributed fashion; and e) verifying the query by verifying the previously computed proofs in parallel, and checking for consistency of their public statements, according to the previously executed query decomposition.

12. The system of claim 11, wherein the succinct digest of the table is collision resistant representation of the table.

13. The system of claim 12, wherein the succinct digest of the table is computed using a Merkle tree.

14. The system of claim 12, wherein the succinct digest of the table is computed using an order-agnostic hash function.

15. The system of claim 12, wherein the succinct digest encodes metadata related to an attribute name of the table or an indexing of the table.

16. The system of claim 11, wherein the table is a relational table and the database is a relational database.

17. The system of claim 11, wherein verifying the query comprises verifying, for each basic query, an input and output to the statement is consistent.

18. The system of claim 11, wherein a size of a proof for verifying the query is dependent on a number of the one or more basic queries.

19. The system of claim 11, wherein a size of a proof for verifying the query is not dependent on a size of the table.

20. The system of claim 11, wherein computing and verifying the one or more proofs for the one or more basic queries comprises using Map/Reduce circuits for Reckle trees.

21. One or more non-transitory computer-readable storage media encoded with instructions executable by one or more processors to provide an application for cryptographically verifying SQL queries, the application comprising: a) a software module computing a succinct digest for a table stored in a database; b) a software module receiving a query in structured query language (SQL) for querying data in the table; c) a software module decomposing the query into one or more basic queries, wherein each of the one or more basic queries comprises a verifiable SQL operator from a library of verifiable SQL operators, and wherein the verifiable SQL operator is a Succinct Non-interactive Argument of Knowledge (SNARK) proof for a statement involving a succinct digest of a table; d) a software module computing, in parallel and in a distributed fashion, one proof per basic query; and e) a software module verifying the query by: i) verifying the previously computed proofs in parallel, and ii) checking for consistency of public statements, according to the previously executed query decomposition.

22. A computer-implemented method for cryptographically verifying SQL queries, the method comprising: a) computing a succinct digest for a table stored in a database; b) receiving a query in structured query language (SQL) for querying data in the table; c) decomposing the query into one or more basic queries; d) computing one proof per basic query in parallel and in a distributed fashion; and e) verifying the query by verifying the previously computed proofs in parallel, and checking for consistency of their public statements, according to the previously executed query decomposition; wherein a size of a proof for verifying the query is not dependent on a size of the table.

23. The method of claim 22, wherein the succinct digest of the table is collision resistant representation of the table.

24. The method of claim 23, wherein the succinct digest of the table is computed using a Merkle tree.

25. The method of claim 23, wherein the succinct digest of the table is computed using an order-

agnostic hash function.
26. The method of claim 23, wherein the succinct digest encodes metadata related to an attribute name of the table or an indexing of the table.
27. The method of claim 22, wherein the table is a relational table and the database is a relational database.
28. The method of claim 27, wherein verifying the query comprises verifying, for each basic query, an input and output to the statement is consistent.
29. The method of claim 22, wherein a size of a proof for verifying the query is dependent on a number of the one or more basic queries.
30. The method of claim 22, wherein computing and verifying the one or more proofs for the one or more basic queries comprises using Map/Reduce circuits for Reckle trees.
31. A computer-implemented system comprising at least one processor and instructions executable by the at least one processor to cause the at least one processor to perform a method for cryptographically verifying SQL queries by performing operations comprising: a) computing a succinct digest for a table stored in a database: b) receiving a query in structured query language (SQL) for querying data in the table; c) decomposing the query into one or more basic queries; d) computing one proof per basic query in parallel and in a distributed fashion; and e) verifying the query by verifying the previously computed proofs in parallel, and checking for consistency of their public statements, according to the previously executed query decomposition; wherein a size of a proof for verifying the query is not dependent on a size of the table.
32. The system of claim 31, wherein the succinct digest of the table is collision resistant representation of the table.
33. The system of claim 32, wherein the succinct digest of the table is computed using a Merkle tree.
34. The system of claim 32, wherein the succinct digest of the table is computed using an order-agnostic hash function.
35. The system of claim 32, wherein the succinct digest encodes metadata related to an attribute name of the table or an indexing of the table.
36. The system of claim 31, wherein the table is a relational table and the database is a relational database.
37. The system of claim 36, wherein verifying the query comprises verifying, for each basic query, an input and output to the statement is consistent.
38. The system of claim 31, wherein a size of a proof for verifying the query is dependent on a number of the one or more basic queries.
39. The system of claim 31, wherein computing and verifying the one or more proofs for the one or more basic queries comprises using Map/Reduce circuits for Reckle trees.
40. One or more non-transitory computer-readable storage media encoded with instructions executable by one or more processors to provide an application for cryptographically verifying SQL queries, the application comprising: a) a software module computing a succinct digest for a table stored in a database; b) a software module receiving a query in structured query language (SQL) for querying data in the table; c) a software module decomposing the query into one or more basic queries; d) a software module computing, in parallel and in a distributed fashion, one proof per basic query; and e) a software module verifying the query by: i) verifying the previously computed proofs in parallel, and ii) checking for consistency of public statements, according to the previously executed query decomposition; wherein a size of a proof for verifying the query is not dependent on a size of the table.